

Legacy Chatbot with System Message

1. Introduction

In older LangChain applications, a chatbot could be created by combining:

Chat Model

+

Conversation Memory

+

Prompt

↓

ConversationChain

For this purpose, older applications commonly used:

ConversationBufferMemory

ConversationBufferWindowMemory

ConversationChain

These are legacy/deprecated APIs, but they are still useful for understanding older LangChain code.

2. Legacy Chatbot Architecture

A simple older chatbot architecture was:

User Input



ConversationChain



Prompt

+

Memory

+

Model



AI Response

ConversationChain helped connect the model and conversation memory.

3. Legacy Chatbot with System Message

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from langchain_classic.chains import ConversationChain
from langchain_classic.memory import
ConversationBufferMemory
```

```
from langchain_core.prompts import (
    ChatPromptTemplate,
    MessagesPlaceholder
)
```

```
# =====
```

```
# CREATE MODEL
```

```
# =====
```

```
llm = ChatOpenAI(
    model="gpt-4o-mini",
    api_key=os.getenv("OPENAI_API_KEY")
)
```

```
# =====
```

```
# CREATE MEMORY
```

```
# =====
```

```
memory = ConversationBufferMemory(  
    memory_key="history",  
    return_messages=True  
)
```

```
# =====
```

```
# CREATE PROMPT WITH SYSTEM MESSAGE
```

```
# =====
```

```
prompt = ChatPromptTemplate.from_messages(  
    [  
        (  
            "system",  
            """"  
  
            You are a Python teacher.
```

Explain everything in very simple English.

Assume the student is a beginner.

Give simple examples wherever possible.

"""

),

MessagesPlaceholder(

variable_name="history"

),

(

"human",

"{input}"

)

]

)

=====

CREATE CONVERSATION CHAIN

```
# =====
```

```
conversation = ConversationChain(  
    llm=llm,  
    memory=memory,  
    prompt=prompt  
)
```

```
print("Legacy AI Chatbot")  
print("Type 'exit' to stop")  
print("-----")
```

```
# =====
```

```
# CHAT LOOP
```

```
# =====
```

```
while True:
```

```
user_input = input("\nYou: ")

if user_input.lower() == "exit":

    print("Chat ended.")
    break

result = conversation.invoke(
    {
        "input": user_input
    }
)

print(
    "AI:",
    result["response"]
)
```

4. Main Components

This program contains four important parts:

1. Model
2. Memory
3. Prompt
4. ConversationChain

5. Create the Model

```
llm = ChatOpenAI(  
    model="gpt-4o-mini",  
    api_key=os.getenv("OPENAI_API_KEY")  
)
```

This is the chat model used to generate the response.

Concept:

Prompt + History



Chat Model



AI Response

6. Create the Legacy Memory

```
memory = ConversationBufferMemory(  
    memory_key="history",  
    return_messages=True  
)
```

This creates legacy conversation memory.

The important options are:

```
memory_key="history"
```

```
return_messages=True
```

7. What Is `memory_key="history"`?

The prompt needs to know where the stored conversation should be inserted.

We use:

```
memory_key="history"
```

and later:

```
MessagesPlaceholder(  
    variable_name="history"  
)
```

Both use the same name:

history

This connects the memory with the prompt placeholder.

8. Easy Connection

ConversationBufferMemory

memory_key="history"



MessagesPlaceholder

variable_name="history"

The names should match.

9. Why return_messages=True?

We use:

return_messages=True

because the prompt contains:

```
MessagesPlaceholder(  
    variable_name="history"  
)
```

So the history is supplied as message objects.

Conceptually:

HumanMessage

AIMessage

HumanMessage

AIMessage

10. Create the Prompt

```
prompt = ChatPromptTemplate.from_messages(  
    [  
        (  
            "system",  
            """  
            You are a Python teacher.  
  
            Explain everything in very simple English.  
            Assume the student is a beginner.  
            Give simple examples wherever possible.  
            """,  
        ),  
    ],
```

```
MessagesPlaceholder(  
    variable_name="history"  
),  
  
(  
    "human",  
    "{input}"  
)  
]  
)
```

The prompt contains three important sections:

System Message



Conversation History



Current Human Input

11. Prompt Architecture

SystemMessage

"You are a Python teacher..."



MessagesPlaceholder("history")



Previous Conversation



Human Message

"{input}"



Model

12. Role of the System Message

The system message gives the chatbot its general behaviour.

Example:

You are a Python teacher.

Explain everything in very simple English.

Assume the student is a beginner.

Give simple examples wherever possible.

So the system instruction defines how the chatbot should respond.

13. Role of MessagesPlaceholder

```
MessagesPlaceholder(  
    variable_name="history"  
)
```

This is the place where previous conversation history is inserted.

Concept:

Prompt

```
|  
|— System Instruction  
|  
|— Previous History  
|  
|— Current User Input
```

14. Role of {input}

```
(  
    "human",  
    "{input}"  
)
```

This represents the current user message.

When we call:

```
conversation.invoke(  
    {  
        "input": user_input  
    }  
)
```

the value of user_input is placed into:
{input}

15. Create ConversationChain

```
conversation = ConversationChain(  
    llm=llm,  
    memory=memory,  
    prompt=prompt
```

)

This older abstraction combines:

Model

+

Memory

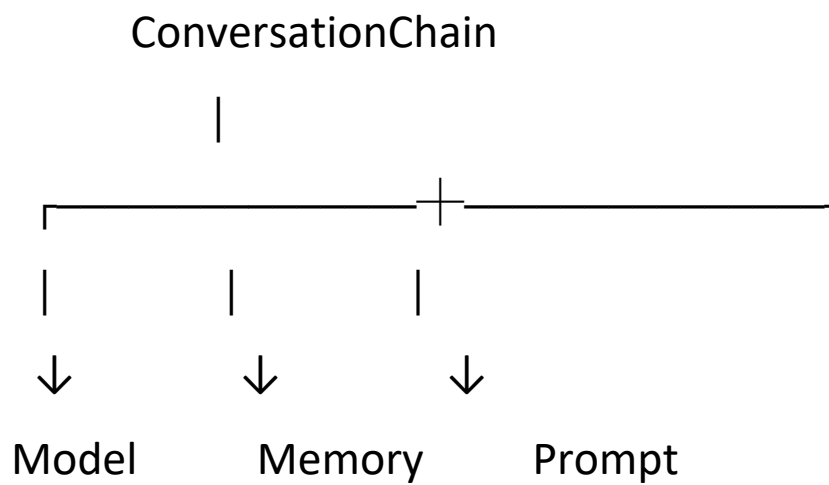
+

Prompt

↓

ConversationChain

16. Main Legacy Architecture



The chain handles the conversation using these components.

17. Calling the ConversationChain

```
result = conversation.invoke(  
    {  
        "input": user_input  
    }  
)
```

The input key is:

input

The chain uses the current input along with its configured memory and prompt.

18. Reading the Response

The program prints:

```
print(  
    "AI:",  
    result["response"]  
)
```

The result contains the generated response under:

response

So:

```
result["response"]
```

gives the final chatbot answer.

19. Chat Loop

while True:

keeps the chatbot running continuously.

Flow:

Read User Input



Check "exit"



conversation.invoke(...)



Get Response



Display Response



Repeat

20. Complete Legacy Conversation Flow

User Input



{input}



ConversationChain



Load Memory



Build Prompt



System Message

+

Previous History

+

Current Input



Model



AI Response



Save Interaction to Memory



Return Response

21. Legacy Memory Cycle

The basic legacy memory cycle can be remembered as:

LOAD



USE



SAVE

LOAD

Previous history is loaded.

USE

The model uses that history with the current input.

SAVE

The new interaction is added back to memory.

22. Compare with Manual History

Earlier, we manually wrote logic similar to:

Add HumanMessage



Send History



Get AIMessage



Add AIMessage

With legacy memory APIs, much of this work was handled automatically.

23. Manual vs Legacy Memory

Manual Approach

Application Code



Store History



Update History



Pass History

Legacy Memory Approach

ConversationChain

+

Memory

↓

Much of History Handling

Done Automatically

24. Why Legacy Memory Was Popular

Legacy memory became popular because it reduced the amount of manual conversation-history code.

Main advantages included:

1. Less manual history code
2. Automatic save/load of conversation history
3. Easy chatbot demonstrations
4. Simple full-buffer memory
5. Simple recent-window memory
6. Easy integration with ConversationChain

Conclusion

Legacy memory made chatbot development simple.

25. Why Was ConversationChain Convenient?

Without it, developers may have to manually connect:

Prompt

+

History

+

Model

ConversationChain provided one abstraction that combined them.

Model

+

Memory

+

Prompt



ConversationChain

This made beginner chatbot examples very easy.

26. Why Not Prefer It for New Applications?

There are several limitations.

Reason 1 — Too Much Automatic History Handling

A lot of history management happens automatically.

This can make the behaviour less explicit.

27. Reason 2 — Multiple Users and Sessions

Modern applications may have:

User A

User B

User C

Each may need separate conversation history.

This requires more flexible session management.

28. Reason 3 — Tools and Agents

Modern applications may involve:

LLM

+

Tools

+

Agents

+

State

+

Persistence

Older memory abstractions are not ideal for all these complex workflows.

29. Reason 4 — Modern LangChain Uses Runnables

Modern LangChain development focuses more on:

LCEL

+

Runnables

+

Explicit Message History

+

Session-Aware State

So the newer architecture is more flexible.

30. Modern Direction

Modern applications may use concepts such as:

RunnableWithMessageHistory

along with:

Session ID

Thread ID

Explicit Message History

Persistent State

31. Legacy vs Modern Direction

Legacy Style

ConversationChain

+

Legacy Memory

Modern Direction

Runnable

+

Explicit Message History

+

Session / Thread Awareness

32. Why Learn Legacy APIs?

Students may still see these APIs in:

- Old videos
- Old courses
- Blogs
- GitHub projects
- Interviews
- Existing company code

Therefore:

Learn legacy APIs so you can understand older LangChain code.

33. Evolution of Conversation Memory

Manual History



Legacy Memory APIs



ConversationChain



Modern Runnable-Based History

34. ConversationBufferMemory

Main idea:

ConversationBufferMemory



Keep Complete Conversation History

As conversation grows:

More History



More Context



Potentially More Tokens

35. ConversationBufferWindowMemory

Main idea:

ConversationBufferWindowMemory



Keep Recent Conversation Window

It reduces older context.

But it can also lose useful older information.

36. Buffer vs Window

Concept	Buffer Memory	Window Memory
Main idea	Full history	Recent history
Old information Kept		May fall outside window
Context usage	Can grow	More controlled
Risk	Large context	Lost old context

37. Memory Is Application-Side State

Very important:

Memory

≠

Model Training

The memory belongs to the application-side conversation mechanism.

It does not mean that the LLM permanently learns the user's conversation.

38. save_context()

Legacy memory provides:

`memory.save_context(...)`

This represents saving one interaction.

Concept:

User Input

+

AI Output



`save_context()`



Memory

39. load_memory_variables()

Legacy memory provides:

`memory.load_memory_variables({})`

This represents retrieving memory information.

Concept:

Memory



load_memory_variables()



Stored Conversation

40. ConversationChain

ConversationChain combined:

Model

+

Prompt

+

Conversation Memory

for older conversational applications.

It reduced manual chatbot code.

41. Important Connection

ConversationBufferMemory



Stores History

ConversationChain



Uses History with Model and Prompt

Together, they made legacy chatbot creation simple.

42. Important Key Points

1. Legacy LangChain introduced Memory APIs to reduce manual history-management code.
2. Memory is application-side state; it is not permanent knowledge inside the LLM.
3. ConversationBufferMemory stores the complete conversation buffer.
4. Full buffer history can grow large and consume more model context.
5. ConversationBufferWindowMemory keeps a limited recent conversation window.
6. Window memory reduces old context but can lose important older information.
7. `save_context()` represents saving an input/output interaction into memory.
8. `load_memory_variables()` represents retrieving memory information.
9. ConversationChain combined a model with conversational memory in older LangChain.

10. The basic legacy memory cycle is:

LOAD



USE



SAVE

11. ConversationBufferMemory, ConversationBufferWindowMemory, and ConversationChain are legacy/deprecated APIs and are not the preferred architecture for new LangChain applications.

12. The modern direction is explicit message history, Runnable-based composition, and session/thread-aware history management.

43. Common Mistake — Thinking Memory Means LLM Memory

Wrong idea:

ConversationBufferMemory



LLM permanently remembers everything

No.

The correct understanding is:

Application Memory



Conversation Context



Provided to Model

44. Common Mistake — Thinking ConversationChain Trains the Model

Wrong.

ConversationChain

≠

Model Training

It simply helps combine conversational components.

45. Common Mistake — Thinking Legacy APIs Are Preferred for All New Apps

Wrong.

These APIs are useful for understanding:

Older Tutorials

Older Projects

Existing Code

For new applications, modern Runnable-based conversation-history approaches are preferred.

46. MCQ 1

What is the main idea of:

`ConversationBufferMemory()`

- A. Keep full conversation history
- B. Generate embeddings
- C. Parse JSON
- D. Run tasks in parallel

Answer: A

Conclusion

Buffer memory represents complete conversational history.

47. MCQ 2

What is the main purpose of:

`ConversationBufferWindowMemory()`

- A. Store API keys
- B. Keep a recent conversation window
- C. Create tools
- D. Generate structured output

Answer: B

Conclusion

Window memory limits the active history to a recent conversational portion.

48. MCQ 3

Which statement is correct today?

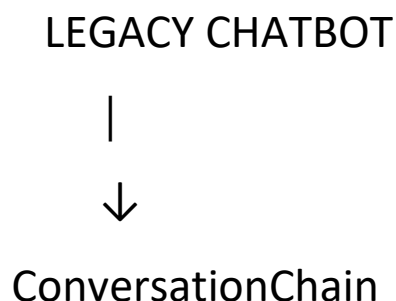
- A. ConversationChain is the preferred architecture for every new LangChain application.
- B. ConversationChain is a legacy/deprecated API useful for understanding older applications.
- C. ConversationChain trains the LLM.
- D. ConversationChain permanently stores all users.

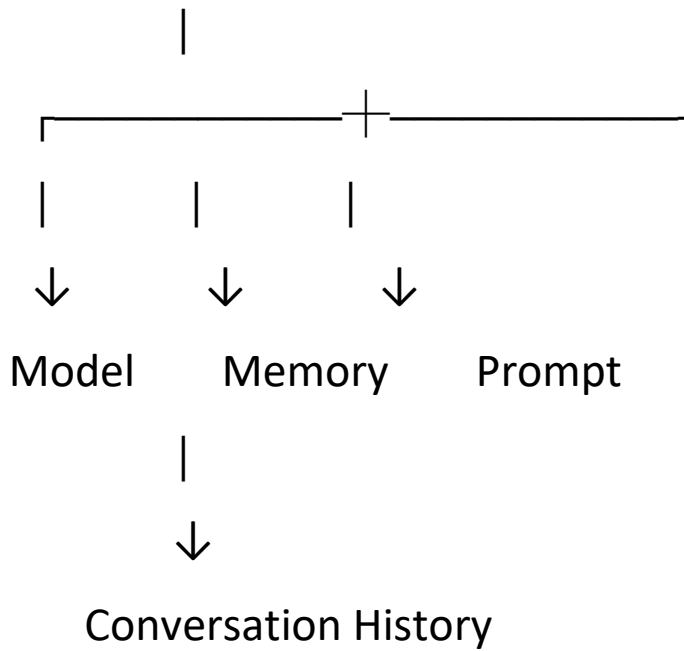
Answer: B

Conclusion

Learn ConversationChain to understand legacy code; learn the modern Runnable-based history approach for new applications.

49. Master Legacy Architecture





50. Prompt Architecture

System Message



Conversation History



Current Human Input



Model



AI Response

51. Final Evolution Diagram

MANUAL HISTORY



`messages.append(...)`



LEGACY MEMORY



ConversationBufferMemory

ConversationBufferWindowMemory

ConversationChain



MODERN DIRECTION



Runnables

Explicit Message History

Session / Thread Management

52. Easy Memory Formula

ConversationBufferMemory



FULL HISTORY

ConversationBufferWindowMemory



RECENT HISTORY

ConversationChain



MODEL + MEMORY + PROMPT

53. One-Line Conclusion

Legacy LangChain used ConversationBufferMemory, ConversationBufferWindowMemory, and ConversationChain to simplify conversational history management, while modern LangChain prefers more explicit Runnable-based, session-aware history management.